Advanced Hardware Engineering

Lab 04: Partial reconfiguration with MicroBlaze

Prof. Dr.-Ing. Ali Hayek

12.11.2026

Goals and learning objectives

Goal

Exchange a hardware module at runtime while the MicroBlaze keeps running.

You will learn

Static and reconfigurable regions, Pblock floorplan, DFX wizard, partial bitstreams, programming.

Deliverables

Block design, floorplan screenshot, bitstream file sizes, terminal output before and after the swap.

Concept: static region and reconfigurable modules

Static region

Never changes. Here: MicroBlaze, memory, UART, GPIO.

Reconfigurable partition

A reserved rectangle (pblock) on the chip with fixed ports.

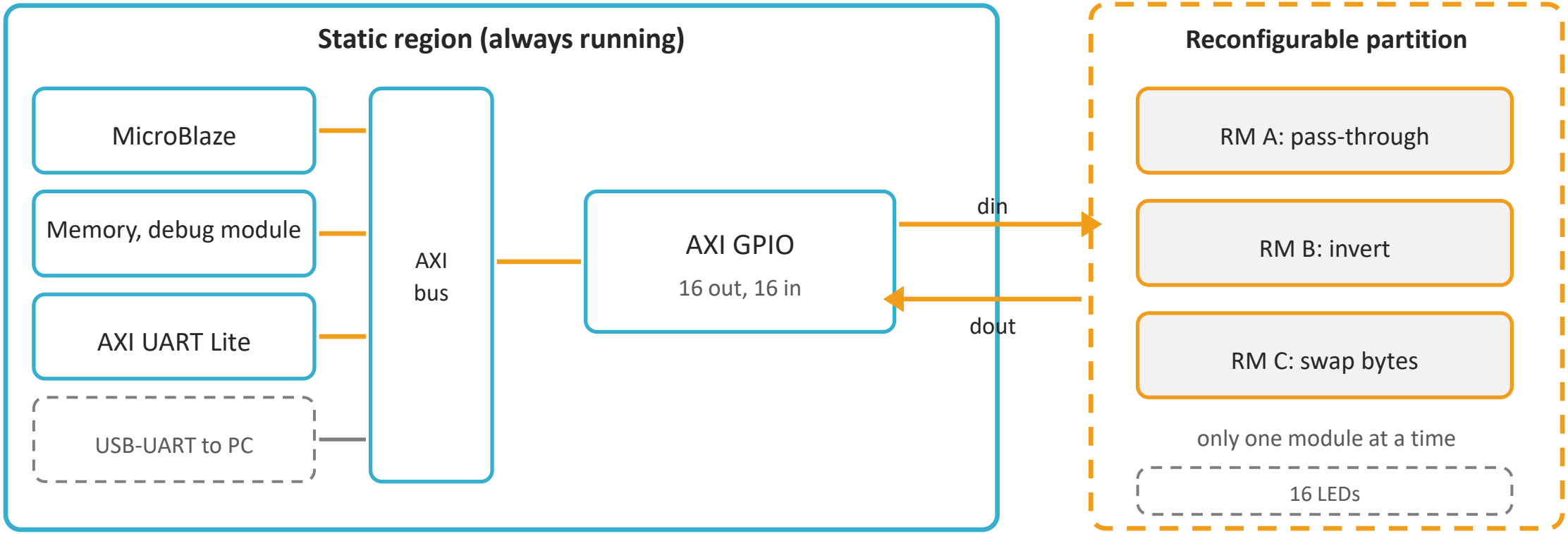
Reconfigurable module

One implementation for the partition: same ports, different function.

Hint

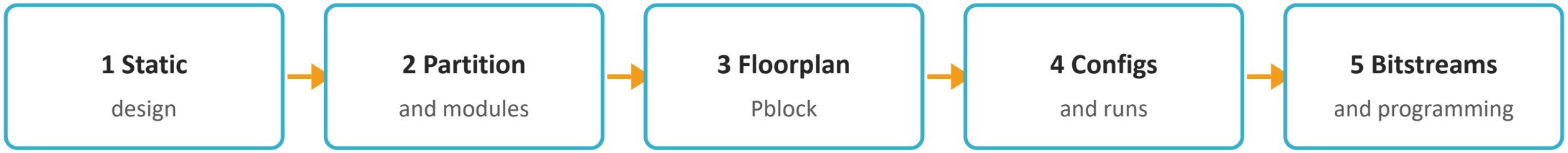
A partial bitstream configures only the partition and is much smaller than a full bitstream. The static region keeps running during the exchange.

What we will build



Blue: static region. Orange, dashed: reconfigurable partition with three possible modules.

The DFX flow in five steps



Tasks 1 and 2 build the design; Tasks 3 and 4 use it

Hint

Every configuration contains the same static region, placed and routed identically. Only the contents of the reserved rectangle differ.

Design rules for reconfiguration

Rule	Why
Same ports for all modules	The static logic around the partition cannot change
Clock from the static region	The module has no clock of its own
Pblock large enough for the biggest module	Every module must fit into the same rectangle
No static logic inside the Pblock	The area is overwritten on reconfiguration
Do not use the outputs during the swap	They are undefined until the module is loaded (decoupling)

Prerequisites and preparation

Item	Details
Lab 02	MicroBlaze system with UART (steps known from that lab)
Board	Nexys A7 with USB cable (power, JTAG, UART)
Tools	Vivado and Vitis of the same release: [release]; check that DFX is available in your license tier
Paths	Short project and workspace paths, e.g. C:\lab\pr and C:\ws\pr
Files	rm_a.vhd, rm_b.vhd, rm_c.vhd, top.vhd, main.c from the instructor
Time	About 3 hours or two sessions

Task 1: build the static design

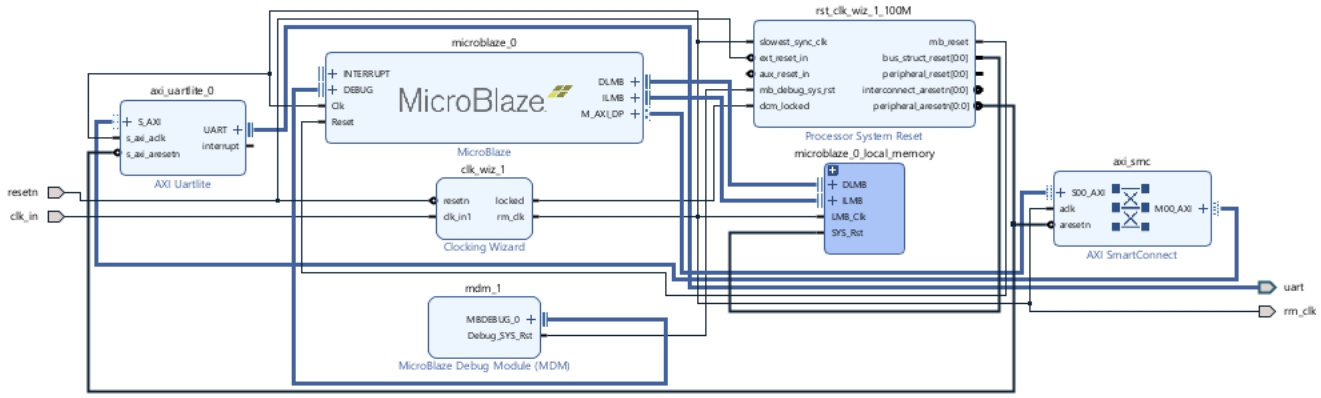
1. Create the project and the MicroBlaze block design
2. Add the AXI GPIO for the module (16 bits out, 16 bits in)
3. Make the ports external and name them
4. Validate and create the HDL wrapper
5. Add the module sources, the top-level file and the constraints

Hint

In Task 1 you build everything that stays fixed. In Task 2 you reserve the exchangeable rectangle and create the module variants.

1.1 Create the project and block design

1. Create a new RTL project with the Nexys A7 board (short path)
2. Create a block design and add the MicroBlaze
3. Run Block Automation (128 KB memory, interrupt controller not needed)
4. Add AXI UART Lite and run Connection Automation
5. Check the diagram against Lab 02

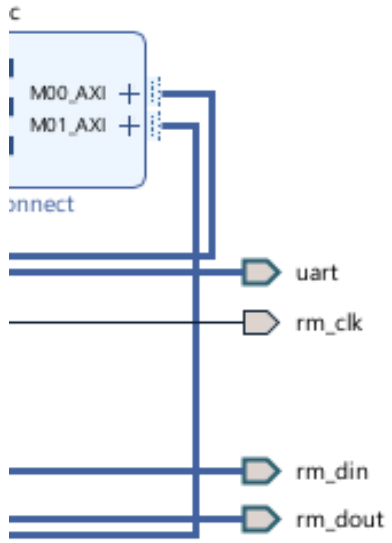


Hint

The static region is the same system as in Lab 02. If you still have that project, you can copy it and continue from there.

1.2 Add the AXI GPIO for the module

1. Add an AXI GPIO and open its customization
2. Enable Dual Channel
3. Channel 1: width 16, All Outputs (data to the module)
4. Channel 2: width 16, All Inputs (data from the module)
5. Run Connection Automation

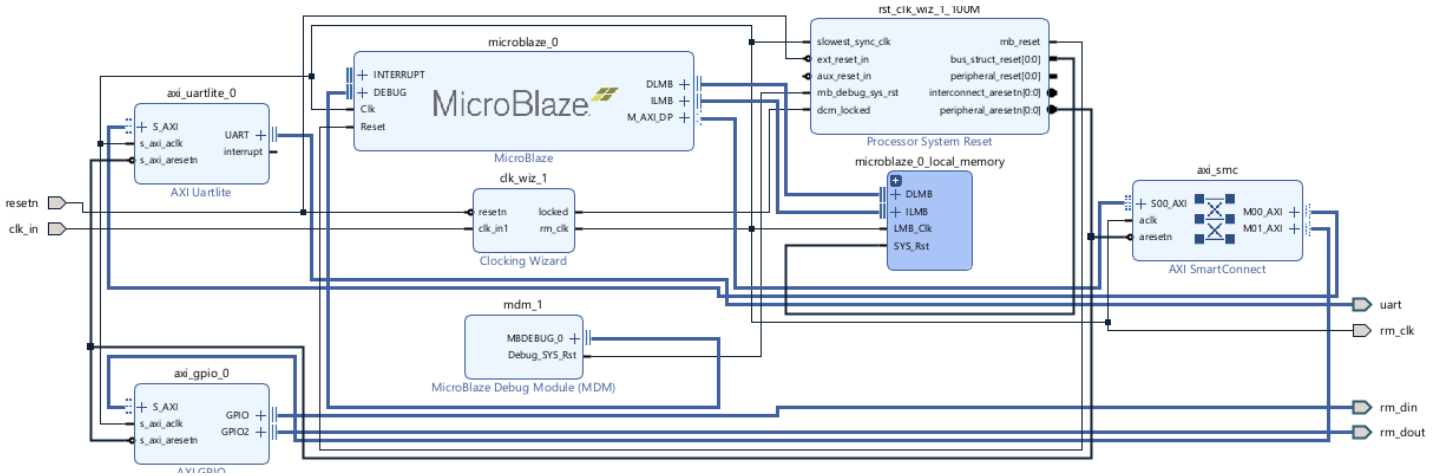


Hint

One channel sends data into the reconfigurable partition and the other reads the result back. Software only needs to read and write registers.

1.3 Make the ports external and name them

1. Make the GPIO output (gpio_io_o) and input (gpio2_io_i) pins external
2. Rename them rm_din and rm_dout (port properties)
3. Make the output of the clock module external and name it rm_clk
4. Name the board ports clk_in, resetn, uart_rx and uart_rx and uart_tx



Hint

The names must be exactly those used in top.vhd. The reconfigurable module receives its clock from the static system.

1.4 Validate and create the HDL wrapper

1. Validate the design
2. Create the HDL wrapper (let Vivado manage it)
3. Open the wrapper file and compare its port names with top.vhd
4. Note the exact name of the wrapper entity (e.g. design_1_wrapper)

Hint

The wrapper is not the top module in this lab. It becomes a component inside top.vhd, next to the reconfigurable module.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VCOMPONENTS.ALL;
entity design_1_wrapper is
  port (
    clk_in : in STD_LOGIC;
    resetn : in STD_LOGIC;
    rm_clk : out STD_LOGIC;
    rm_din : out STD_LOGIC_VECTOR ( 15 downto 0 );
    rm_dout : in STD_LOGIC_VECTOR ( 15 downto 0 );
    uart_rx : in STD_LOGIC;
    uart_tx : out STD_LOGIC
  );
end design_1_wrapper;
```

1.5 The three module sources

```
-- rm_a: pass-through
entity rm_a is
  port (clk  : in  std_logic;
        din  : in  std_logic_vector(15 downto 0);
        dout : out std_logic_vector(15 downto 0));
end entity;

architecture rtl of rm_a is begin
  process (clk) begin
    if rising_edge(clk) then
      dout <= din;
    end if;
  end process;
end architecture;
```

```
-- rm_b: dout <= not din;
-- rm_c: dout <= din(7 downto 0)
--             & din(15 downto 8);

Same entity ports in all three files;
only the line in the process differs.

Input 0x0001 gives:
  A: 0x0001   pass-through
  B: 0xFFFE   invert
  C: 0x0100   swap bytes
```

Hint

All modules of one partition need identical ports. Add the library lines for ieee.std_logic_1164 above each entity. The software uses the test value 0x0001 to find out which module is loaded.

1.6 The top-level file

```
entity top is
  port (CLK100MHZ    : in  std_logic;
        CPU_RESETN   : in  std_logic;
        UART_TXD_IN  : in  std_logic;
        UART_RXD_OUT : out std_logic;
        LED : out std_logic_vector(15 downto 0));
end entity;
-- components: design_1_wrapper (adapt to your own
design)
--             and rm_a
```

```
u_sys : design_1_wrapper
  port map (clk_in => CLK100MHZ,
    resetn => CPU_RESETN,
    uart_rx => UART_TXD_IN,
    uart_tx => UART_RXD_OUT,
    rm_clk => rm_clk, rm_din => din,
    rm_dout => dout);

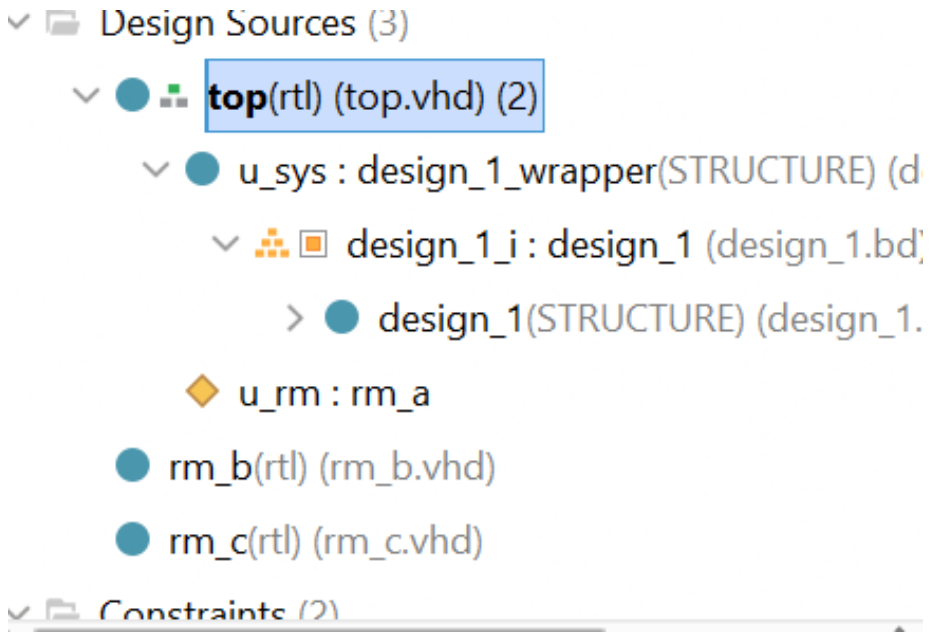
u_rm : rm_a   -- this becomes the partition
  port map (clk => rm_clk, din => din,
            dout => dout);
LED <= dout;
```

Hint

Write the file top.vhd. Adapt the wrapper name and the port names to your design. The instance name u_rm is used for the partition and the Pblock.

1.7 Add the sources and constraints

1. Add rm_a.vhd and top.vhd as design sources (rm_b and rm_c later)
2. Set top as the top module
3. Add the constraints file: clock, reset, UART and the 16 LEDs
4. Check the pins against the Digilent master XDC
5. Run synthesis and look at the hierarchy: u_sys and u_rm



Hint

The constraints use the port names of the master XDC, so you can copy the lines directly. Do not add rm_b and rm_c as normal sources: they become modules of the partition in Task 2.

Task 2: set up the reconfiguration

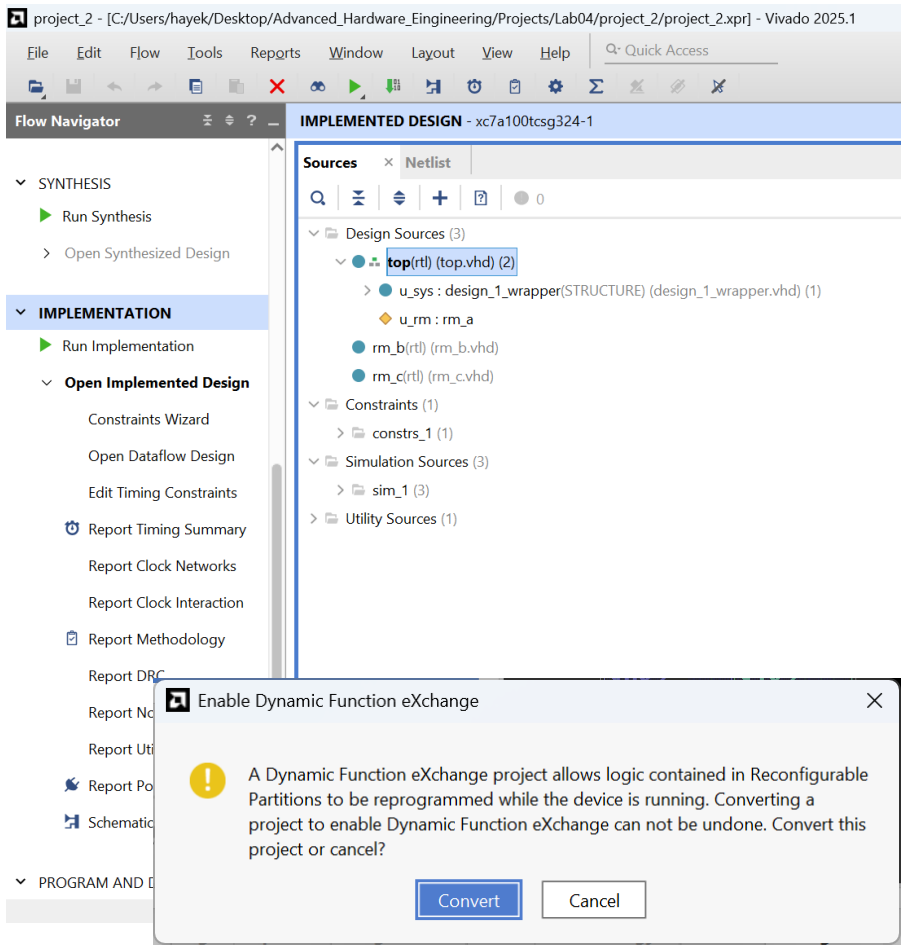
1. Create the partition definition from the instance u_rm
2. Add the modules rm_b and rm_c
3. Draw the Pblock (floorplan) and set its properties
4. Create the configurations and implementation runs (DFX wizard)
5. Generate the bitstreams and inspect the files
6. Export the hardware for Vitis

Hint

Task 2 turns the plain design into a reconfigurable one. The expensive part is the implementation: three configurations run one after another.

2.1 Create the partition definition

1. In the Sources window, hierarchy view: right-click u_rm
2. Select Create Partition Definition
3. Name it rm_pd; the reconfigurable module name is rm_a
4. Confirm: u_rm is now marked as a reconfigurable partition

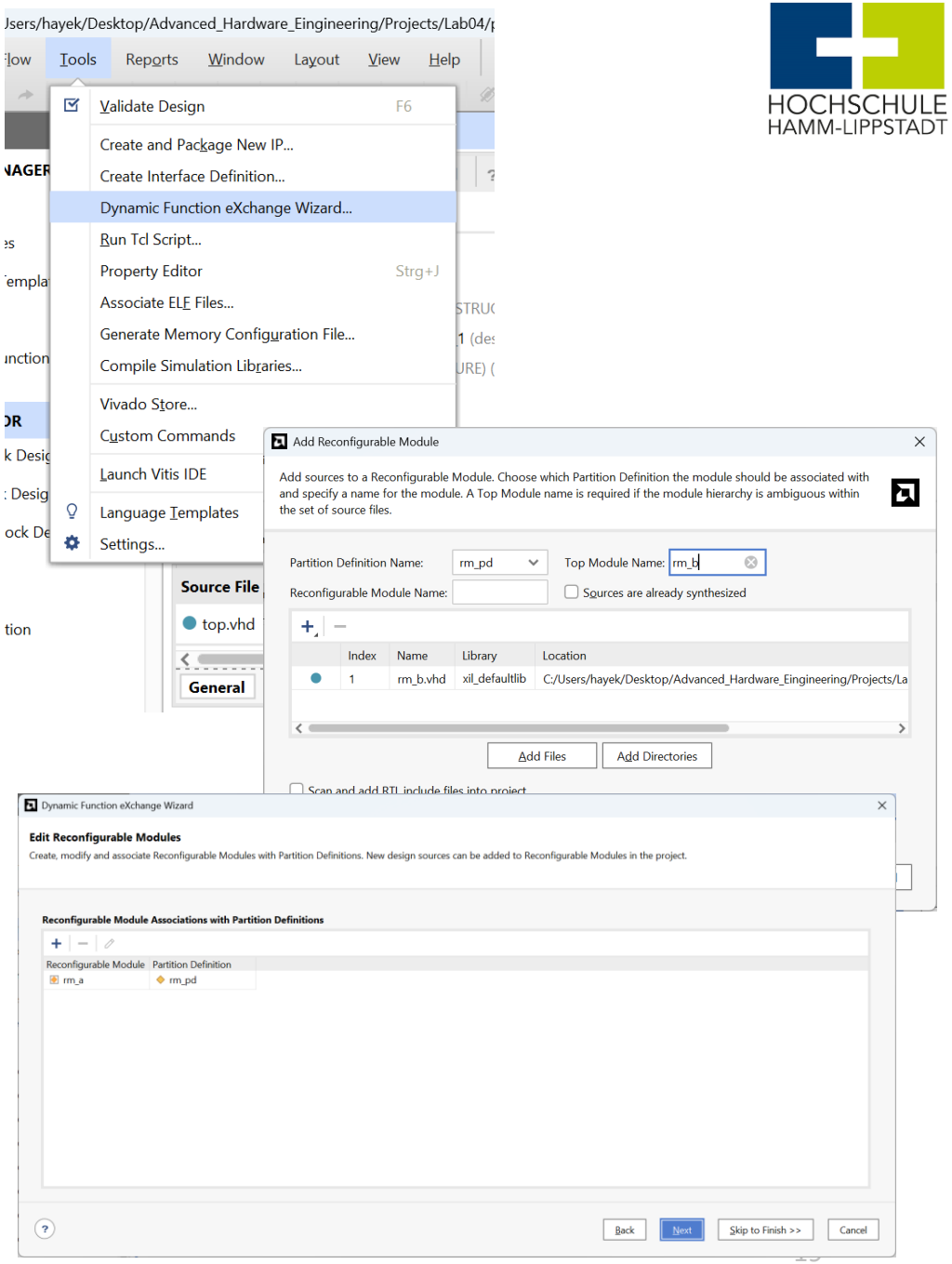


Hint

The partition definition declares the port interface of the partition and the first module. Afterwards Vivado treats u_rm as a black box of the static design.

2.2 Add the modules rm_b and rm_c

1. Open Tools, Dynamic Function eXchange Wizard (or the partition definition)
2. On the page Edit Reconfigurable Modules add a module
3. Add rm_b.vhd with the module name rm_b
4. Repeat for rm_c.vhd and rm_c
5. Check that three modules are listed for rm_pd

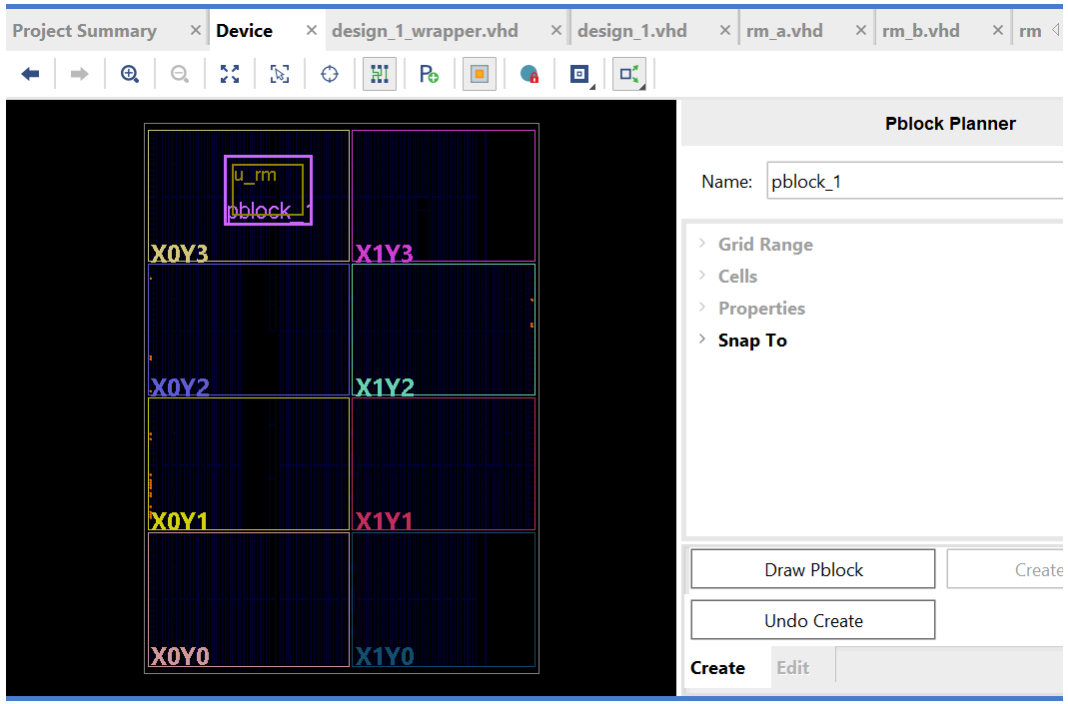


Hint

All modules of one partition must have the same ports. Vivado checks this when you add them.

2.3 Draw the Pblock (floorplan)

1. Run synthesis and open the Synthesized Design
2. Open the Device window
3. Select the cell u_rm in the netlist
4. Draw a rectangle with Floorplanning, Draw Pblock
5. Set RESET_AFTER_RECONFIG and SNAPPING_MODE in the Pblock properties (next slide)
6. Save the constraints



Hint

The Pblock is the reserved rectangle. It must be larger than the largest module and aligned to the configuration frames. Snapping mode lets Vivado adjust the corners automatically.

2.3 Draw the Pblock (floorplan)

How to set RESET_AFTER_RECONFIG and SNAPPING_MODE in the Pblock properties

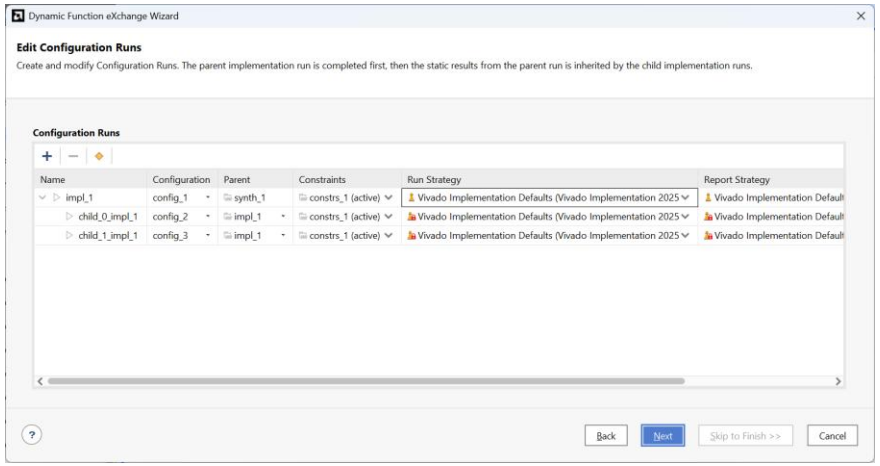
Type in the tcl console:

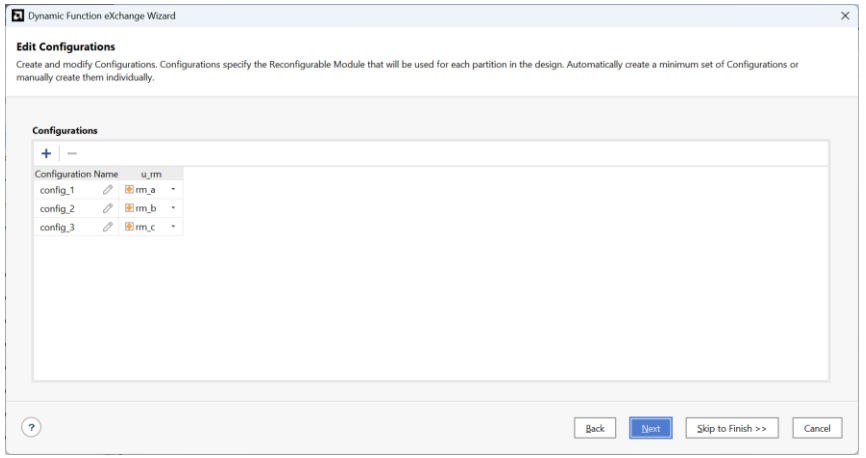
```
set_property RESET_AFTER_RECONFIG true [get_pblocks pblock_1]

set_property SNAPPING_MODE ON [get_pblocks pblock_1]
```

2.4 Configurations and runs (DFX wizard)

1. Open the DFX wizard, page Edit Configurations
2. Create one configuration per module (config_a, config_b, config_c)
3. Page Edit Configuration Runs: create the parent and child runs
4. Check the summary page





Hint

A configuration is a complete design: the static region plus one module in the partition. The static part is implemented once and reused by the other configurations.

2.5 Implement and generate the bitstreams

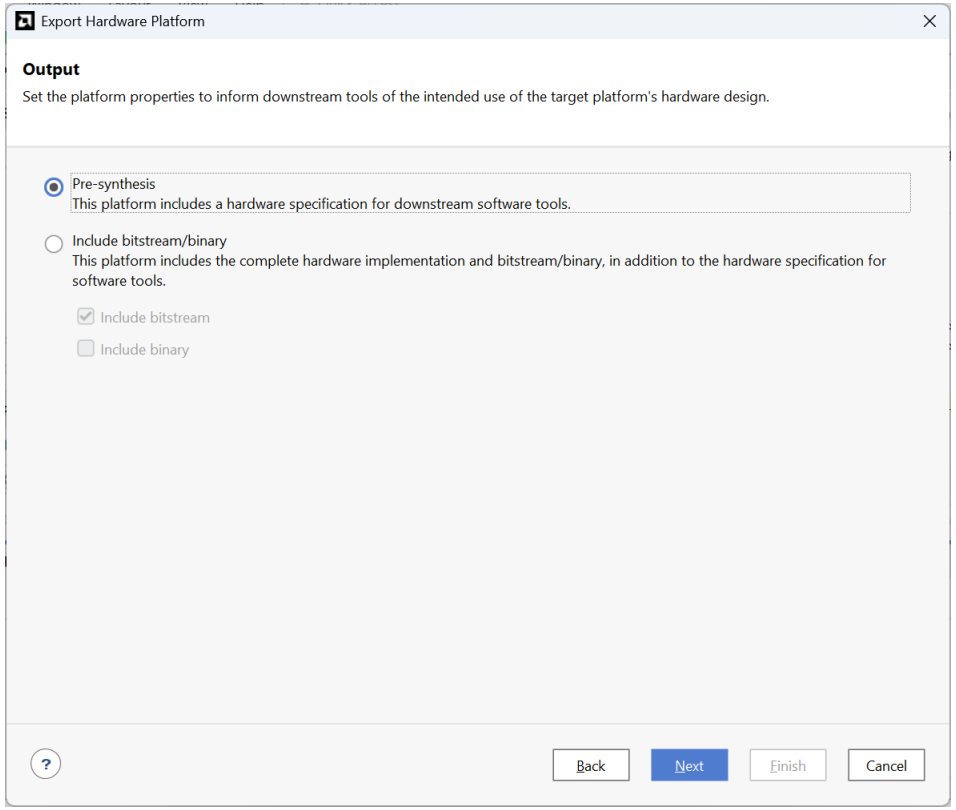
1. Run Generate Bitstream (for all configuration runs)
2. Wait: this takes several minutes
3. Open the folders of the implementation runs
4. Find one full bitstream and the partial bitstreams (names contain partial)
5. Compare the file sizes

Hint

The full bitstream contains the static region and module A. Each partial bitstream contains only the rectangle of the partition.

2.6 Export the hardware for Vitis

1. Select File, Export, Export Hardware
2. Choose pre-synthesis (without bitstream)
3. Save the XSA file in a short path
4. Note the location



Hint

Vitis only needs the address map. The FPGA is programmed from the Hardware Manager, because the partial bitstreams are loaded there.

Task 3: the software

1. Create a platform from the XSA and build it
2. Create an application and paste main.c
3. Set the base address of the GPIO
4. Build the application

Hint

The program does not know which module is loaded. It finds out by sending 0x0001 and looking at the answer: A gives 0x0001, B gives 0xFFFE and C gives 0x0100.

3.1 Platform and application in Vitis

1. Create a new workspace with a short path
2. Create a platform from the XSA of Task 2 and build it
3. Create an application component for this platform
4. Replace main.c by the file from the instructor
5. Set GPIO_BASE to the address in the Address Editor and build

Hint

Short paths avoid the build error you saw in Lab 03. The example address in main.c is only an example: read yours from the Address Editor.

3.2 The program

```
static const char *identify(void)
{
  Xil_Out32(GPIO_BASE + DIN, 0x0001);
  usleep(100);
  u32 r = Xil_In32(GPIO_BASE + DOUT)
          & 0xFFFF;
  if (r == 0x0001) return "RM A";
  if (r == 0xFFFE) return "RM B";
  if (r == 0x0100) return "RM C";
  return "unknown";
}
```

```
int main(void)
{
  u16 cnt = 0;
  while (1) {
    const char *n = identify();
    Xil_Out32(GPIO_BASE + DIN, cnt);
    usleep(1000);
    u32 out = Xil_In32(GPIO_BASE + DOUT)
              & 0xFFFF;
    xil_printf("%s in=%04x out=%04x\r\n",
               n, cnt, out);
    cnt++;  usleep(250000);
  }
}
```

Hint

Use the complete main.c file from moodle: it contains the includes and the defines GPIO_BASE, DIN and DOUT, which are not shown here. Only one main per file.

3.3 What you should see

Module	Function	Output for input 0x0001	LEDs with the counter
RM A	Pass-through	0x0001	Counter on LED0 to LED7 as binary
RM B	Invert all bits	0xFFFE	Inverted counter on all LEDs
RM C	Swap bytes	0x0100	Counter on LED8 to LED15

Task 4: reconfigure at runtime

1. Program the full bitstream of the first configuration (module A)
2. Run the application and open the terminal
3. Load the partial bitstream for module B and watch the terminal
4. Load the partial bitstream for module C, then module A again
5. Answer the questions

Hint

The MicroBlaze is part of the static region: it is not reset and does not stop. Only the partition changes its function.

4.1 Program the full bitstream and start the software

1. In Vivado open the Hardware Manager, Open Target, Auto Connect
2. Right-click the device: Program Device
3. Select the full bitstream of the parent configuration (module A)
4. Run the application from Vitis and open the serial terminal
5. Check: the terminal shows RM A and the LEDs show the counter

Hint

The full bitstream configures the whole chip. Afterwards the static region is running and module A is loaded.

4.2 Load the partial bitstream for module B

1. Leave the application running and the terminal open
2. In the Hardware Manager: Program Device
3. Select the partial bitstream of module B (the name contains partial)
4. Click Program and watch the terminal and the LEDs

Hint

This is the key moment: the output changes from RM A to RM B without resetting the MicroBlaze. The messages continue with the same counter.

4.3 Load module C and go back to A

1. Program the partial bitstream of module C
2. Observe the terminal and the LEDs (counter on the upper byte)
3. Program the partial bitstream of module A
4. Compare: is the state exactly as at the start?

Hint

Any module can be loaded in any order. Loading A again restores the original function without loading the full bitstream.

Questions and observations

Question	Hint
How large is the full bitstream and how large are the partial ones?	Compare the file sizes of Task 2
Why do the three partial bitstreams have the same size?	Same rectangle, same number of frames
What happened to the terminal output during the swap?	Look for unknown lines
Why was the MicroBlaze not affected?	It lies in the static region
When would you not use reconfiguration?	Overhead in area, effort and time

Extensions and further ideas (for your project?)

Decoupler

Add a DFX decoupler IP so that the interface is held stable while a module is loaded.

Self-reconfiguration

Add the AXI HWICAP so the MicroBlaze loads the partial bitstream from memory.

Larger modules

Use your PWM IP from Lab 01 or a filter as a module with an AXI interface.

Deliverables and summary

Item	Content
Design	Screenshot of the block design and of the floorplan with the Pblock
Bitstreams	List of the full and partial bitstream files with their sizes
Terminal	Output before and after each swap
Questions	Answers to the questions of Task 4
Summary	Static region, partition, modules, partial bitstreams: what each is for

Thank you for your attention!

Prof. Dr. -Ing. Ali Hayek

University of Applied Sciences Hamm-Lippstadt (HSHL)

Tel.: **+49 (2381) 8789 – 924**

E-Mail.: **ali.hayek@hshl.de**